МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна.

Студент гр. 4343 _____ Маркашов Г.Д.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Цель Работы.

Изучение алгоритмов нахождения расстояния редактирования (расстояния Левенштейна) и восстановления последовательности операций трансформации.

Задание.

Вариант 11.

Среди всех префиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти префиксы. Среди всех суффиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти суффиксы. Асимптотическая сложность решения задачи должна быть не больше, чем при обычном вычислении редакционного расстояния.

Теоретический минимум

Расстояние Левенштейна (редактирующее расстояние) — это метрика сходства между двумя последовательностями (строками), определяемая как минимальное количество операций вставки, удаления и замены одного символа, необходимых для превращения одной строки в другую. Задача нахождения этого расстояния является классической задачей динамического программирования. Она основывается на том, что решение для строк длины n, m можно получить, зная решения для их префиксов меньшей длины.

Выполнение работы.

В данной лабораторной работе используется алгоритм Вагнера-Фишера.

Алгоритм Вагнера-Фишера используется для вычисления редакционного расстояния между двумя строками с учётом заданных весов операций (вставка, удаление, замена) с помощью таблицы, в которой рассматриваются все возможные префиксы первой и второй строки. Данная таблица постепенно заполняется, шаг за шагом вычисляя минимальную стоимость преобразования для всё более длинных частей строк. Когда таблица полностью заполнена, значение в правой нижней ячейке и есть искомая минимальная стоимость преобразования первой строки во вторую целиком.

Программа на языке программирования Python реализовывает алгоритм Вагнера-Фишера.

def lev_dist_prefixes(s1: str, s2: str, mode="префиксов") – функция по поиску расстояние левинштейна для всех префиксов s1 относительно s2. Функция возвращает список расстояний.

def main() – функция точка входа в программу

Работа алгоритма

Функция lev_dist_prefixes принимает на вход оригинальную строку и строку до которой нужно посчитать расстояние.

n, m – длина строк s1, s2 соответственно

prev – список значений предыдущей строчки матрицы используемой алгоритмом Вгнера-Фишера.

curr – список текущих значений строчки матрицы.

Примечание: В текущей реализации используется не вся матрица целиком, а только ее 2 строчки. Это сделано для оптимизации скорость и памяти.

dists – список расстояний для префиксов s1 до s2.

Основной цикл идет от 1 до n(включительно) и на каждой итерации формирует список curr используя следующие формулы для последовательного заполнения curr:

1) Если буквы совпадают по позиции, то выполняется условие s1[i-1] == s2[j-1] и записывается в текущую позицию curr значение из prev[j-1].
2) Если буквы не одинаковые на позиции, то выбирается минимум из:
    a. prev[j] + 1. Соответствует удалению элемента из строки
    b. curr[j-1] + 1. Соответствует вставки элемента в строку
    c. prev[j-1] + 1. Соответствует замене элемента в строке.

Итогом основного цикла становиться позиция m списка curr. Именно на этой позиции находиться расстояние Левенштейна для конкретного префикса до s2. Цикл выполняется ровно n раз, что соответствует всем возможным префиксам строки s1.

Сложность алгоритма

Временная сложность: внешний цикл итерируется n раз, внутренний проходит по всем символам префикса по всей строке s2, что соответствует m. Так как это внутренний цикл, то он исполниться ровно n раз, следственно сложность алгоритма составит O(N*M).

Пространственная сложность: prev и curr массивы занимают m + 1 места. Массив dists занимает ровно n + 1 значений. Как следствие сложность O(N+M).

Код выполнения работы см. в Приложении А.

Тестирование см. в Приложении Б.

Приложения

Приложение А

main.py:

```
def lev_dist_prefixes(s1: str, s2: str, mode="префиксов"):
    n, m = len(s1), len(s2)
    prev = list(range(m + 1))
    curr = [0] * (m + 1)

    dists = [prev[m]]

    print(f"\nРасчет для {mode}")
    print(f"Сравниваем '{s1}' и '{s2}'")
    print(f"Начальное состояние (пустой префикс s1): {prev}")

    for i in range(1, n + 1):
        curr[0] = i
        for j in range(1, m + 1):
            if s1[i-1] == s2[j-1]:
                curr[j] = prev[j-1]
            else:
                curr[j] = min(prev[j], curr[j-1], prev[j-1]) + 1

        dists.append(curr[m])
        print(f"Шаг {i}: подстрока '{s1[:i]}' -> расстояние до s2 =
{curr[m]} | Весь ряд: {curr}")

        prev[:] = curr[:]

    return dists

def main():
    s1 = input("Введите строку s1 (оригинал): ")
    s2 = input("Введите строку s2 (цель): ")
    n = len(s1)

    # Считаем префиксы
    prefix_dists = lev_dist_prefixes(s1, s2, mode="префиксов")
    min_pref = min(prefix_dists)
    best_prefixes = [s1[:i] for i, d in enumerate(prefix_dists) if d
== min_pref]

    # Считаем суффиксы (используем реверс)
    s1_rev = s1[::-1]
    s2_rev = s2[::-1]
    suffix_dists_rev = lev_dist_prefixes(s1_rev, s2_rev,
mode="суффиксов")
```

```
    # Разворачиваем результаты обратно
    suffix_dists = [0] * (n + 1)
    for k in range(n + 1):
        suffix_dists[n - k] = suffix_dists_rev[k]

    print("\nИТОГОВЫЙ АНАЛИЗ:")
    print(f"Расстояния для всех префиксов: {prefix_dists}")
    print(f"Минимальное расстояние префикса = {min_pref}")
    print(f"Лучшие префиксы: {best_prefixes}")

    print(f"\nРасстояния для всех суффиксов: {suffix_dists}")
    min_suff = min(suffix_dists)
    best_suffixes = [s1[i:] for i, d in enumerate(suffix_dists) if d
== min_suff]
    print(f"Минимальное расстояние суффикса = {min_suff}")
    print(f"Лучшие суффиксы: {best_suffixes}")

if __name__ == "__main__":
    main()
```

Приложение Б

Входные данные	**Результат выполнения**
error error	Минимальное расстояние суффикса = 0 Лучшие суффиксы: ['error']
error rorre	Минимальное расстояние суффикса = 2 Лучшие суффиксы: ['ror']
a avb	Минимальное расстояние суффикса = 2 Лучшие суффиксы: ['a']
avb v	Минимальное расстояние суффикса = 1 Лучшие суффиксы: ['vb', 'b', '']
qwe zxc	Минимальное расстояние суффикса = 3 Лучшие суффиксы: ['qwe', 'we', 'e', '']